HireFlow

PROEDUVATE · FULL STACK · ROUND 2

RECRUITMENT & APPLICANT TRACKING SYSTEM

Project report: problem understanding, architecture, technology choices, implementation, a page-by-page walkthrough, security, test results, limitations and future work.

Christopher Jesuraj · Rajalakshmi Institute of Technology
Repository: github.com/christhecreator56/hireflow
November 2026

CONTENTS

What's inside

1. Problem understanding
2. Solution overview & roles
3. Architecture & technology choices
4. How it was built: phases & bottleneck analysis
5. End-to-end hiring workflow
6. Page-by-page walkthrough
7. The two USPs in depth
8. Data model
9. Resume parsing & real-resume demo data
10. Security considerations
11. Testing & results
12. Limitations & future enhancements
13. Running the project

21
database models with migrations and seed data

62
documented API operations (OpenAPI 3 / Swagger)

74 / 74
automated tests passing (67 backend + 7 frontend)

4
roles: employer, recruiter, hiring manager, candidate

1 · PROBLEM UNDERSTANDING

What the brief asks for

Organisations need one system to publish vacancies, collect applications, move candidates through recruitment stages, schedule interviews and record structured feedback. Every action must be traceable, and candidate information must be protected.

Requirement	How HireFlow covers it
Employer, Recruiter, Hiring Manager, Candidate roles	Role-based access control in middleware plus a single resource-level access module; hiring managers are scoped to their own jobs in SQL
Job postings & application forms	Job editor with a status machine (draft → published ⇄ paused → closed) and custom questions of five types
Candidate profiles, resume upload, tracking	Candidate portal, S3 resume storage, automatic resume parsing, application timeline
Pipeline with configurable stages	Per-vacancy stage list edited as data, Kanban board, approval gates
Interview scheduling & evaluation forms	Panel scheduling with double-booking checks, 1–5 scorecards per job criterion
Templates & notifications	Editable email templates, sandbox email provider (Mailpit), in-app notifications
Analytics & audit history	Time-to-stage, sources, vacancy status, conversion funnel; append-only audit log
USP: configurable hiring workflow	Stages, notifications and approvals per vacancy, without code changes
USP: candidate data governance	Consent capture, role-limited access, retention, export and deletion workflows

2 · SOLUTION OVERVIEW

Who uses HireFlow

Candidate

Signs up with consent, uploads a resume that is parsed automatically, sees how well they match every role, applies, follows each stage on a timeline, and can export or delete their data.

Recruiter

Creates vacancies and their pipelines, screens and moves candidates on a Kanban board, schedules interviews, sends templated emails and reads analytics.

Hiring manager

Sees only the vacancies they own, sits on interview panels, submits scorecards, and

Employer

Runs the organisation: team and roles, final approvals, retention policy, deletion requests and the full audit trail.

approves stage moves (such as offers) that are gated to them.

How the system is put together

React SPA → /api/v1 (cookie) → Express API → PostgreSQL · S3 storage · Mailpit

The browser talks only to the API. The API owns all business rules and writes to PostgreSQL through Prisma. Resume files go to S3-compatible storage and are read through short-lived presigned links, so files never stream through the API. Emails are rendered from templates and handed to an SMTP sandbox without blocking the request. A nightly job runs the data-retention sweep.

Layer	Choice	Why
Frontend	React 19 · TypeScript · Vite · Tailwind v4	Typed components, fast dev server; the design system lives in CSS variables, which also makes the dark theme possible
Backend	Node.js 22 · Express 5 · TypeScript · Zod	One language across the stack; Express 5 forwards async errors natively; Zod validates every request
Database	PostgreSQL 16 · Prisma	Relational integrity (unique, foreign-key and cascade rules), migrations and seed in one tool, raw SQL for analytics
Object storage	S3 API (SeaweedFS locally)	The same code runs against AWS S3 or Cloudflare R2; MinIO no longer publishes images, so SeaweedFS replaced it
Email	Nodemailer + Handlebars → Mailpit	Real SMTP flow in a sandbox; templates are editable data and HTML-escape their variables
Auth	bcrypt · JWT in httpOnly cookie · RBAC	The token can't be read by page scripts; roles are checked in middleware and records in a central access module
Quality	Vitest · Supertest · Testing Library · Swagger	Integration tests against a real database; API contract served as documentation
DevOps	Docker Compose · GitHub Actions	One command starts the stack; CI runs type checks and tests on every push

Backend design rules

- **Thin routes:** each one parses its input, authorizes, calls `lib/`, writes to the audit log and responds.
- **One way to change a stage:** `moveApplication()`. It checks that the stage belongs to the job, enforces the approval gate, uses an optimistic lock, writes the stage history and sends the notification.
- **One place for permissions:** `lib/access.ts`, which prevents users from opening other people's records by changing an ID.
- **Analytics from history:** `StageTransition` is append-only and every report is a single SQL aggregate.

Phases & bottleneck analysis

Before writing code, we studied similar open-source applicant trackers (notably OpenCATS) and typical PERN job portals for known failure points, and designed against them from day one.

Bottleneck seen elsewhere	Fix built into HireFlow
Candidate list queries examining 300k+ rows for 15 results (10–18 s page loads at 50k candidates)	Lean single-query Kanban data, composite indexes, pagination
Hard-coded pipeline stages	Stages and approval gates per vacancy, stored as data
Analytics computed in application memory	SQL aggregates over the append-only stage history
Email sending blocking the request	Queued email log, sent without awaiting, status updated afterwards
Resumes streamed through the server	5 MB MIME-checked upload to S3; 5-minute presigned downloads
Users reading others' records by changing an ID (IDOR)	Central access module plus tests that prove the 403s
Two recruiters moving the same card	Optimistic lock: a stale move returns 409 "refresh and retry"
Interviewer double-booking	Overlap check on every schedule and reschedule
Personal data kept forever	Consent records, retention sweep, anonymization

Build phases

1. **Scaffold:** repository, Docker services (Postgres, S3 storage, mail sandbox), Prisma schema and migrations.
2. **Auth & RBAC:** login, cookies, roles, access module, audit trail.
3. **Domain APIs:** jobs, candidate portal, pipeline engine, interviews, email, reports, governance.
4. **Frontend:** every screen, built in parallel by a specialised sub-agent against the finished API.
5. **Quality:** 67 backend tests and the API and database docs, each produced by its own sub-agent. Their reviews caught real bugs (a missing auth check, unvalidated query parameters, data requests not scoped to the organisation), which were fixed.
6. **Product polish:** design system, landing page, full-screen auth, dark theme, animations, resume parsing, real-resume demo data.

5 · END-TO-END WORKFLOW

From vacancy to hire

Recruiter publishes job → Candidate signs up + uploads resume → Applies → Screening → Interview + scorecards → Offer approval → Hired

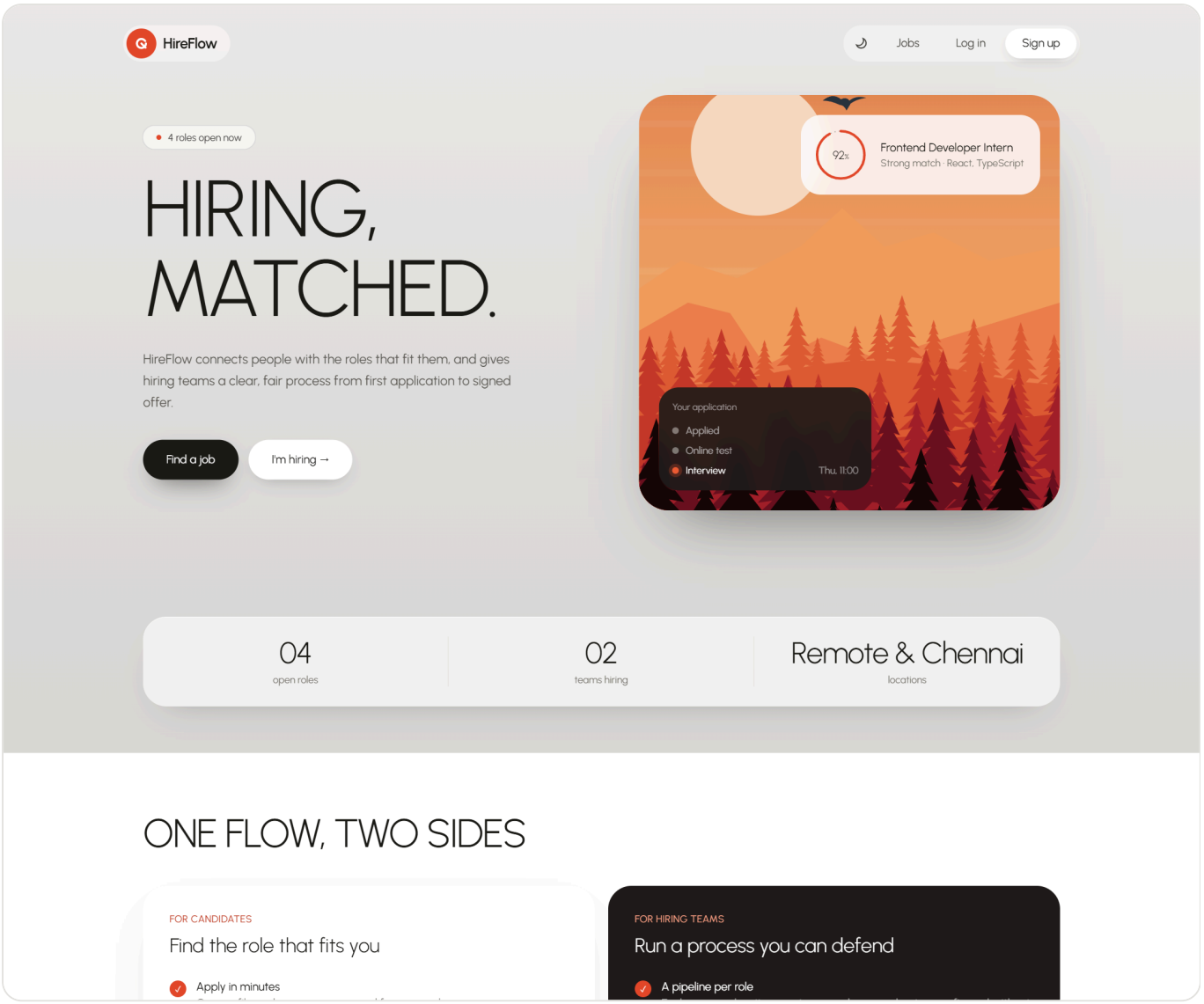
1. **Recruiter** creates a vacancy (fields, skills, questions, and a pipeline from a template or the default) and publishes it. Only published jobs appear publicly and accept applications.
2. **Candidate** signs up. Privacy consent is required and is stored with its policy version and IP address. They upload a PDF resume; the server reads it and fills in skills, phone, city, LinkedIn and headline.
3. The job board ranks every role by **skill match**. The candidate answers the role's questions and applies; an "application received" email goes to the sandbox inbox.
4. **Recruiter** drags the card to Screening. The move is recorded as a stage transition and in the audit log, and a stage-update email is sent if that stage has notifications on.
5. **Recruiter** schedules an interview with a panel. A clash with an existing interview is refused with the busy person's name. The candidate is emailed the time and place.
6. **Panel members** submit scorecards rated 1–5 per job criterion, with a recommendation.
7. The recruiter moves the candidate to **Offer**, which is gated: the API answers 202 and creates a pending approval. The configured approver (hiring manager or employer) approves, the move completes, and the offer email is sent.
8. Moving to **Hired** closes the application. Analytics update from the stage history.
9. Later, the candidate can **export** all their data or request **deletion**. The employer approves, personal data is anonymized, files are deleted, and the hiring statistics stay correct.

Edge cases demonstrated

Edge case	Behaviour
Recruiter moves a candidate into an approval-gated stage	202 Accepted, card shows "Approval pending", candidate stays put until the approver decides
Scheduling an interviewer who is already booked	409 "Double booking: <name> already has an interview at that time"
Deleting a stage that still holds candidates	409 with the number of candidates to move first
Candidate opens another candidate's application or a staff page	403 Forbidden
Applying twice, or applying to a paused job	409 / 400 with a clear message

Public site

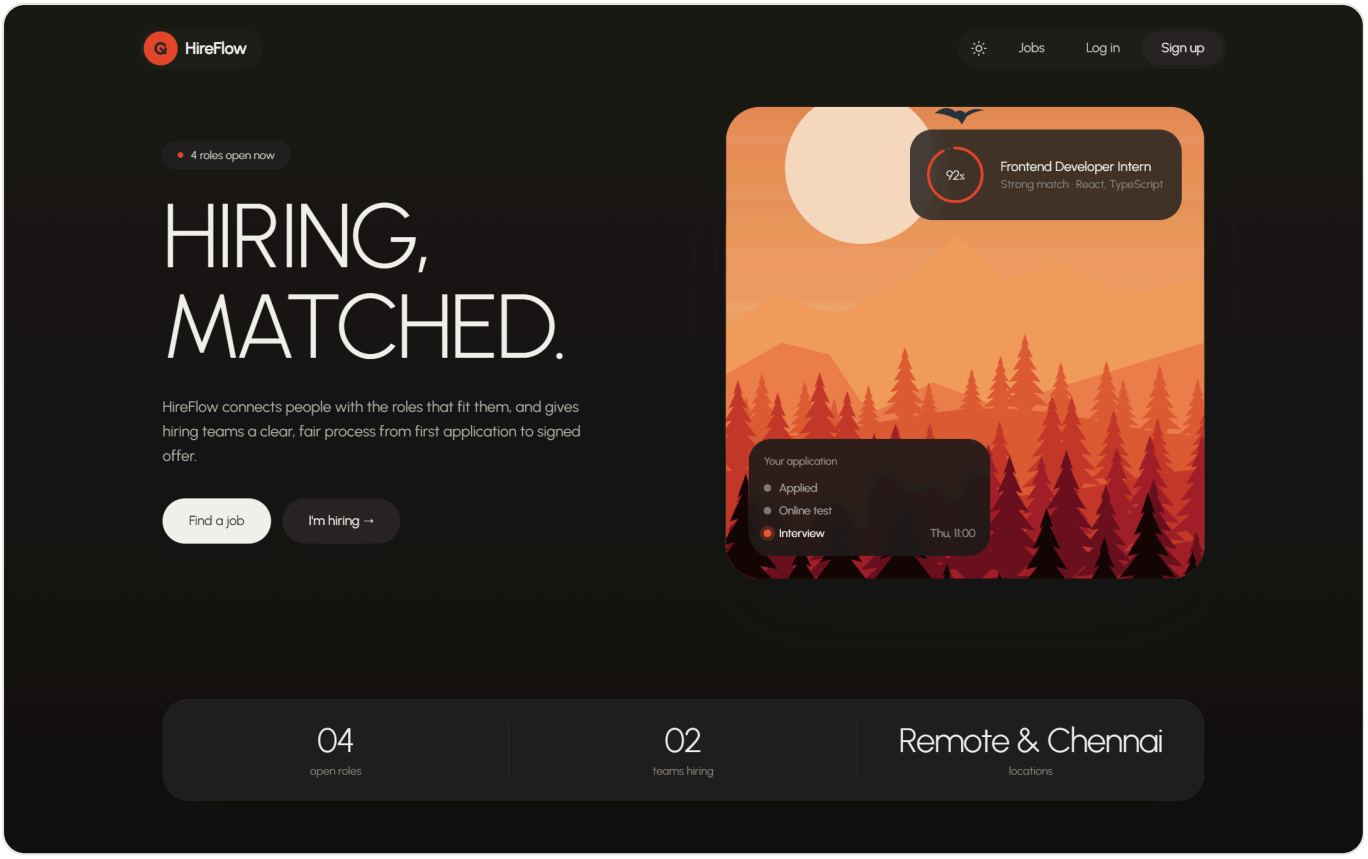
Landing page /



Purpose: the front door for first-time visitors. It explains what HireFlow is and lets each visitor choose a path: *Find a job* or *I'm hiring*.

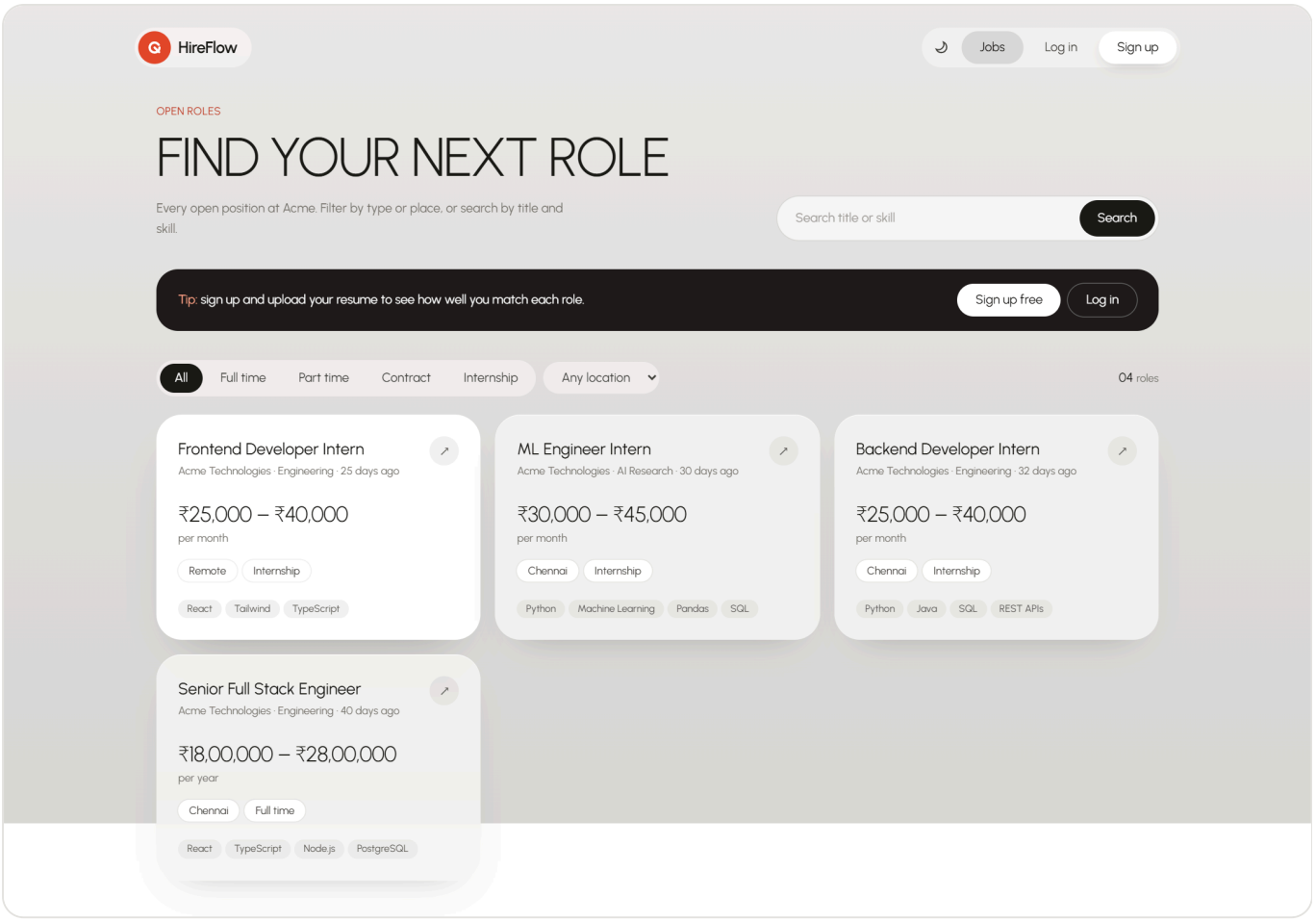
What's on it: a hero with a product preview, live facts computed from the open roles (count, teams, locations), "One flow, two sides" for candidates and hiring teams, a four-step "How it works", three live job cards, and a closing call to action. Sections fade in as you scroll. Signed-in users skip this page and go straight to their workspace.

Dark theme



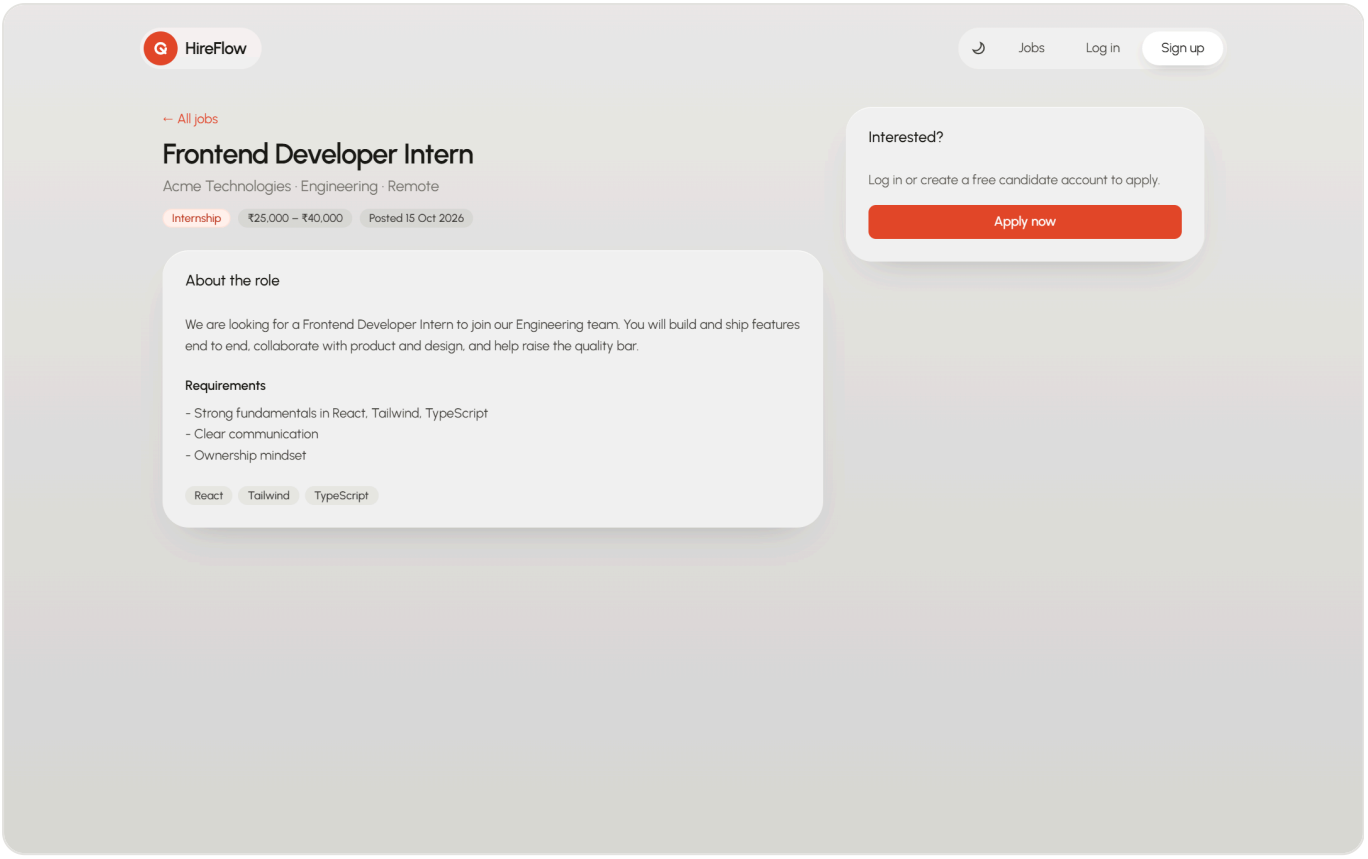
Every page supports light and dark themes through the sun/moon toggle. The choice is remembered, and first-time visitors get their device's setting. Dark mode is implemented by switching the colour variables, so all pages follow automatically. An automatic contrast check passed on 14 pages.

Job board /jobs



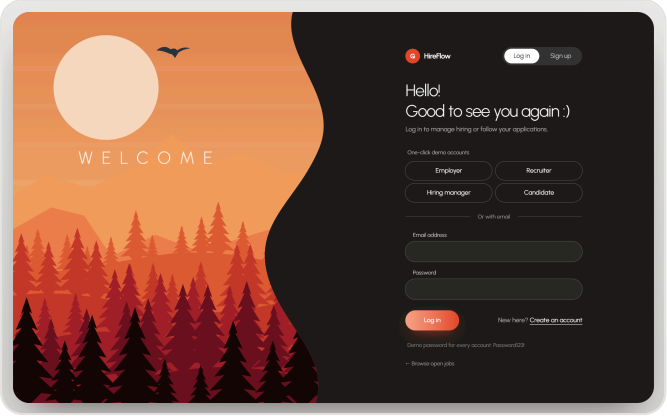
All published vacancies, with keyword search, type filters and a location filter built from the live jobs. Visitors who aren't signed in see salary and role essentials and a tip inviting them to sign up to see match scores.

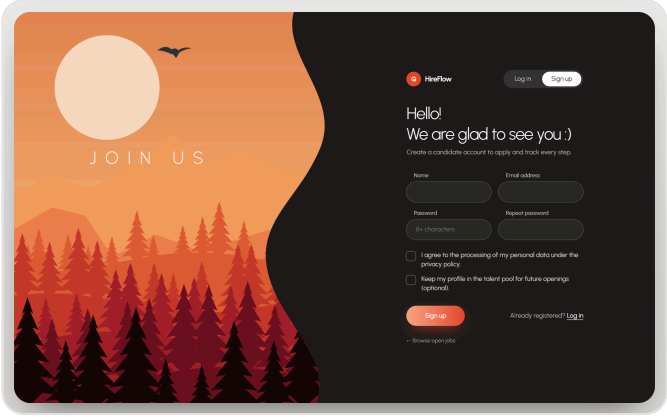
Job detail & application form /jobs/:id



The full description, requirements and skills. Candidates apply here: they answer the role's custom questions (all five types are supported), pick or upload a resume and choose how they heard about the role. Required answers, duplicate applications and paused jobs are all checked on the server.

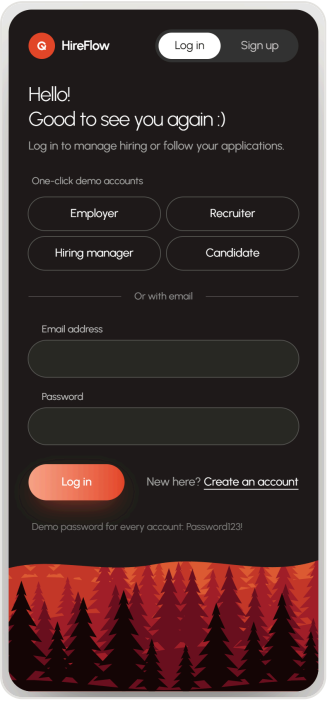
Log in & sign up `/login · /register`

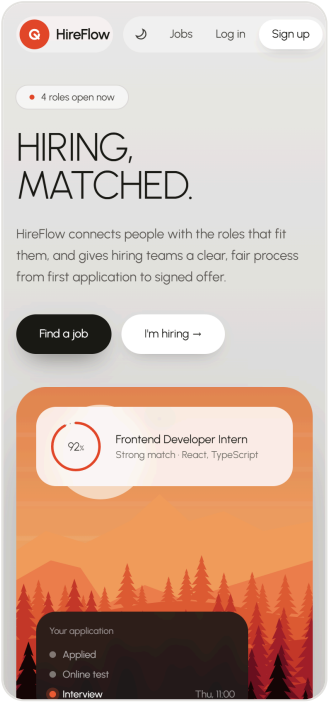




A full-screen split card with an illustrated sunset-forest panel. Log in has one-click demo accounts for every role. Sign up checks that the two passwords match and requires privacy consent, with optional talent-pool consent. Both are stored as consent records with their policy version.

On a phone

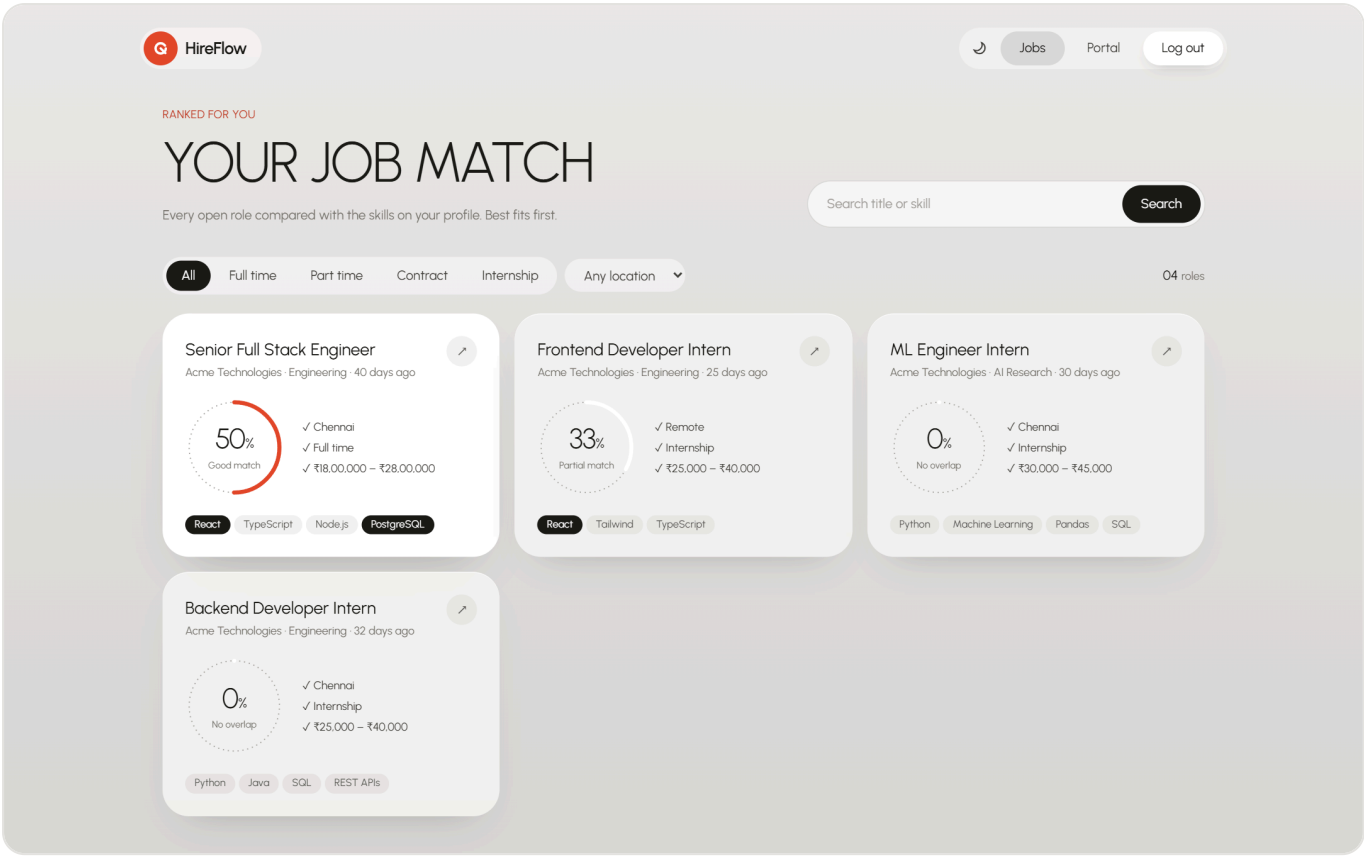




Every page works down to 375 px wide. On phones the auth card puts the form on top with the forest as a band along the bottom, and fits the screen exactly. The staff app's navigation moves into a slide-out menu.

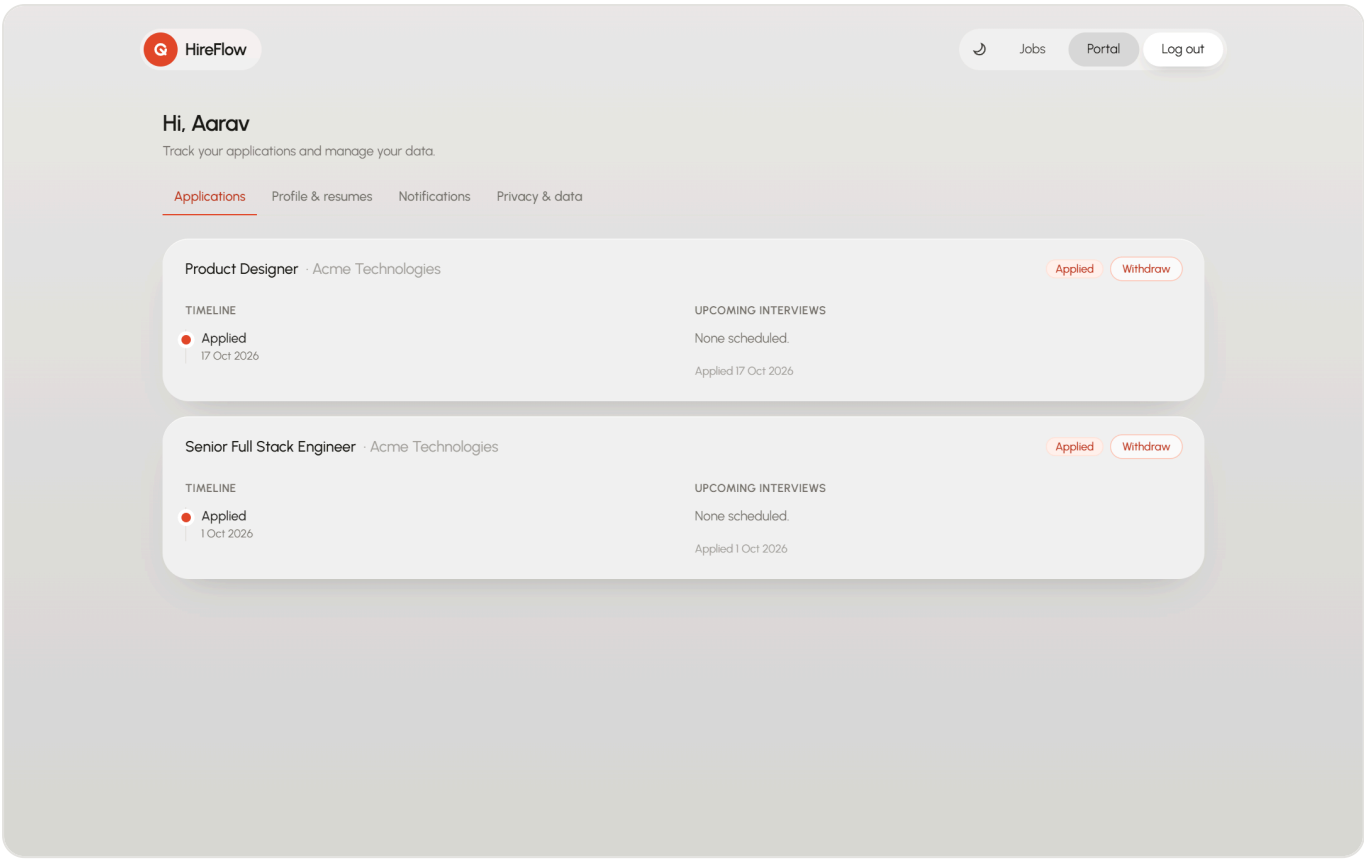
Candidate experience

Job match /jobs as a candidate



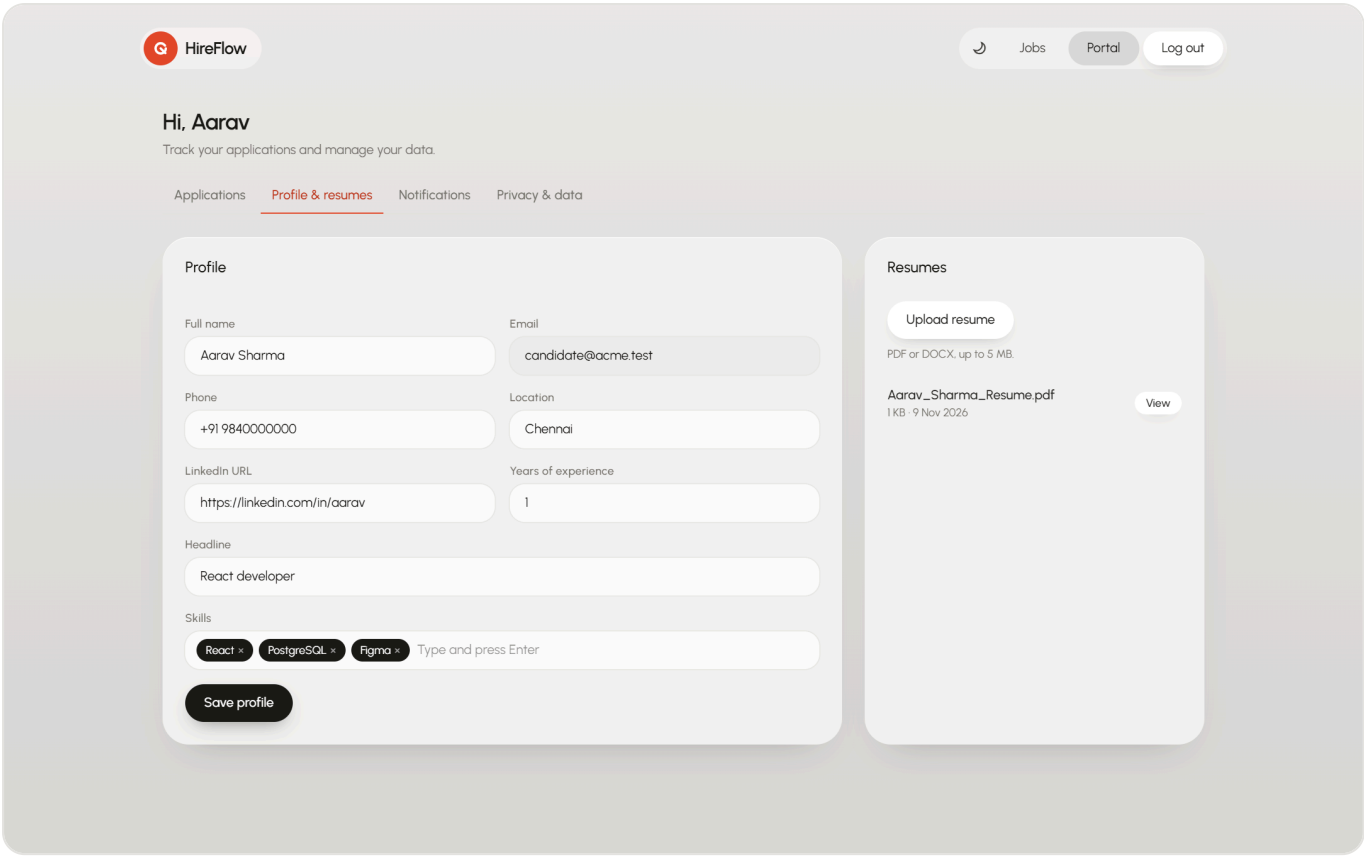
For signed-in candidates the board becomes **"Your job match"**: every role shows a ring with the share of its required skills found on the candidate's profile, the best fits come first, and skills the candidate already has are shown as dark chips.

My applications /portal



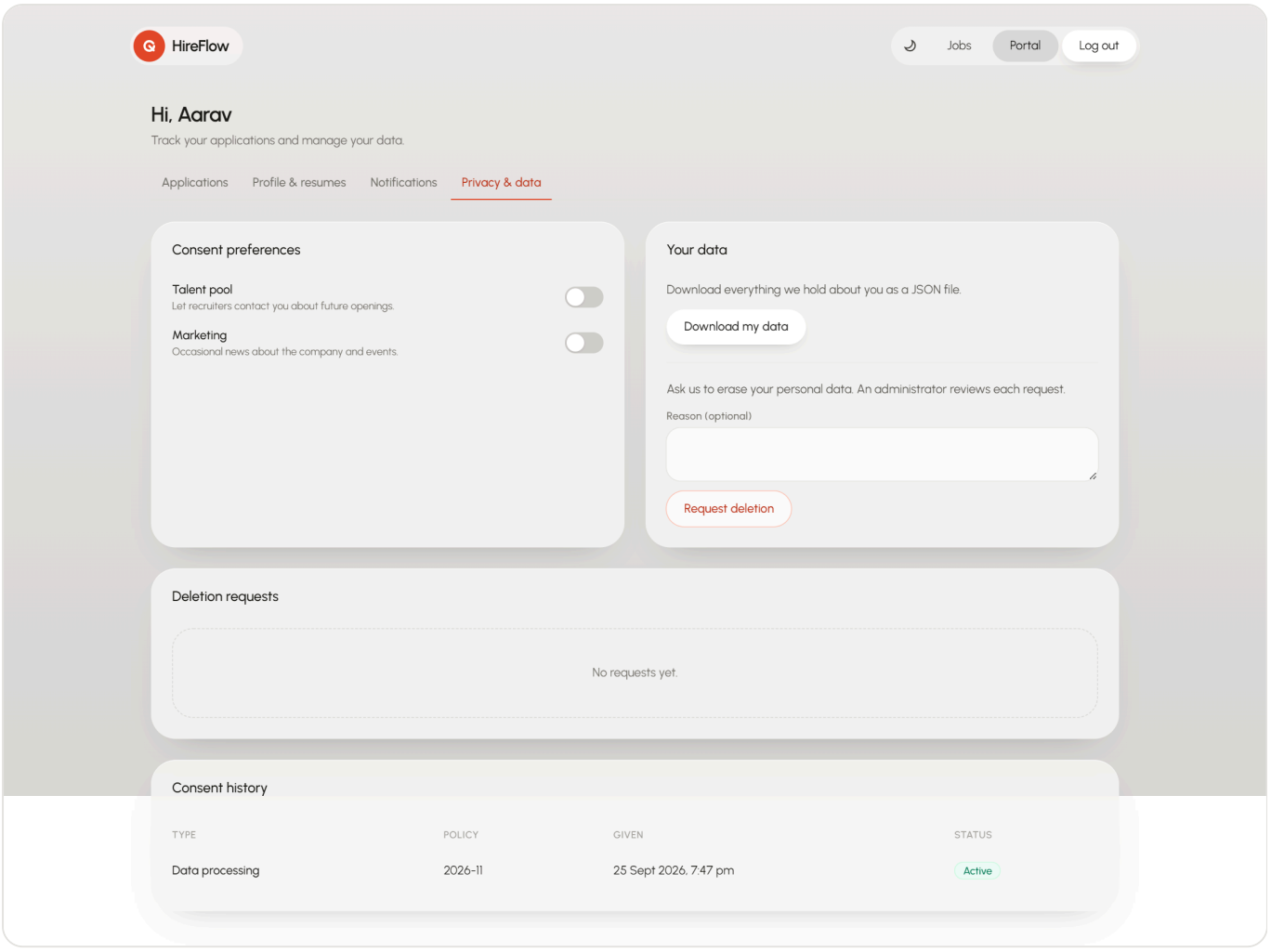
Each application shows a candidate-friendly status (internal stage names are kept internal, and rejections read "Not selected"), a dated stage timeline, upcoming interviews with time and place, and a withdraw button.

Profile & resumes /portal/profile



Candidates edit only the fields they're allowed to edit; the API rejects anything else. Uploading a PDF resume runs the parser: detected skills are added and empty contact fields are filled in, without overwriting anything the candidate typed. Resumes open through 5-minute presigned links.

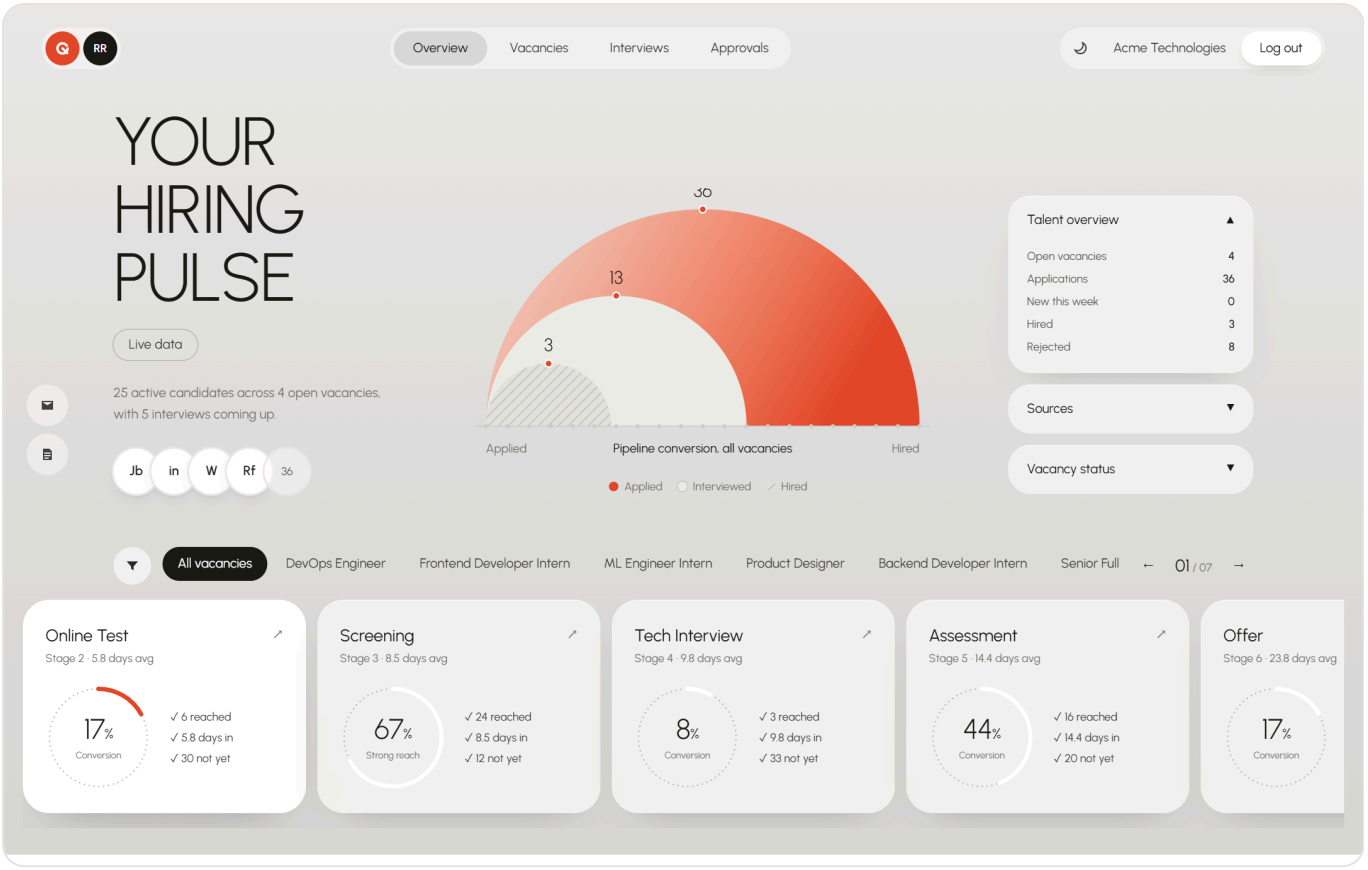
Privacy & data /portal/privacy



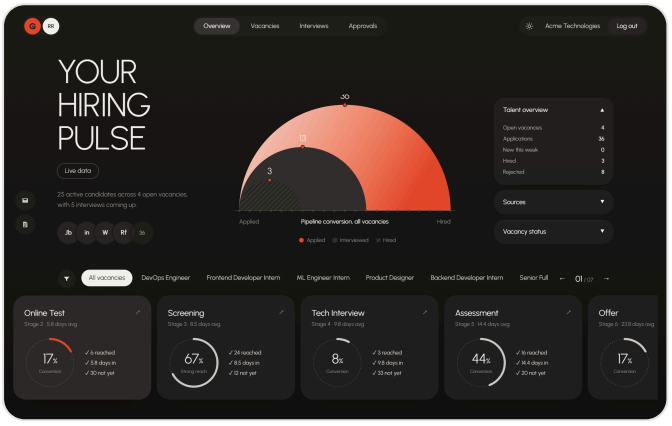
Candidates can grant or withdraw talent-pool and marketing consent (history is kept), download everything held about them as JSON, and request deletion. They can see each request's status as the employer processes it.

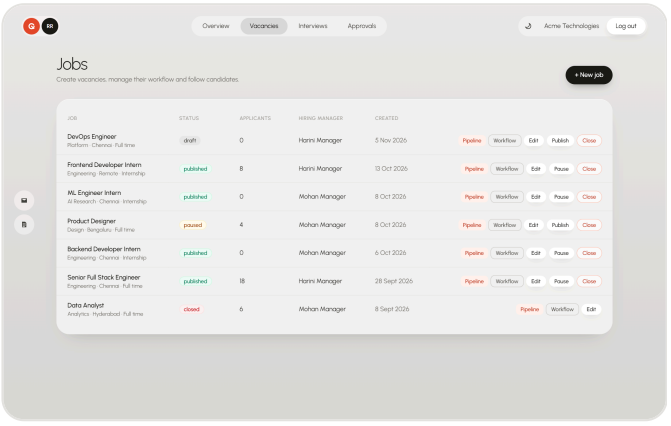
Hiring team

Dashboard: hiring pulse `/dashboard`



Three nested half-discs show applied → interviewed → hired, sized by area. Panels on the right open to show talent overview, sources and vacancy status. Below, a vacancy filter and swipeable stage cards show, for each stage, the share of candidates who reached it, the average days to get there, and how many haven't yet. Hiring managers see only their own vacancies.



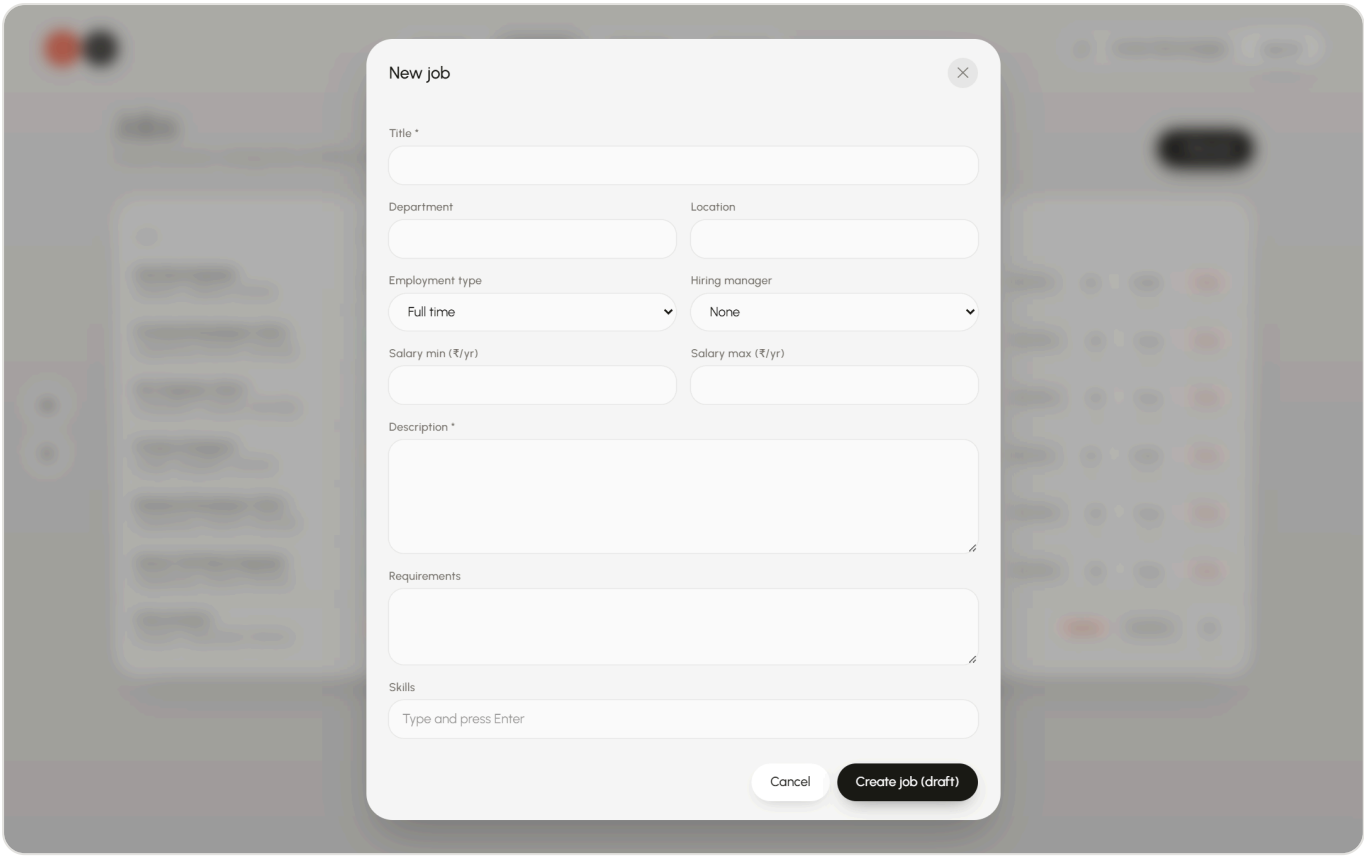


Left: the dashboard in dark mode. Right: the vacancies list.

Vacancies `/jobs-admin`

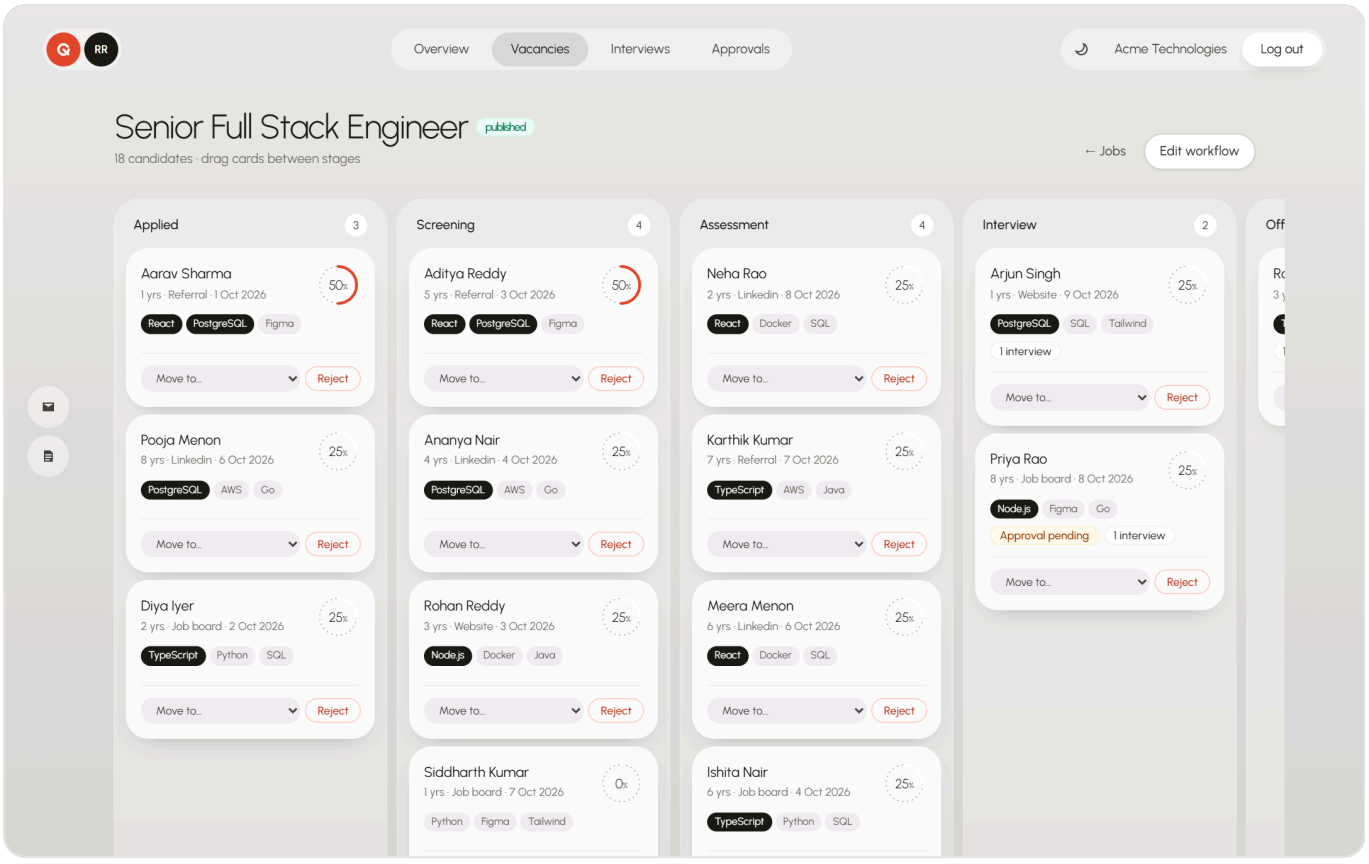
Every vacancy with its status and applicant count. Status buttons show only the legal next states (for example, a closed job can't be republished), and each row links to its pipeline and workflow editor.

Creating a job



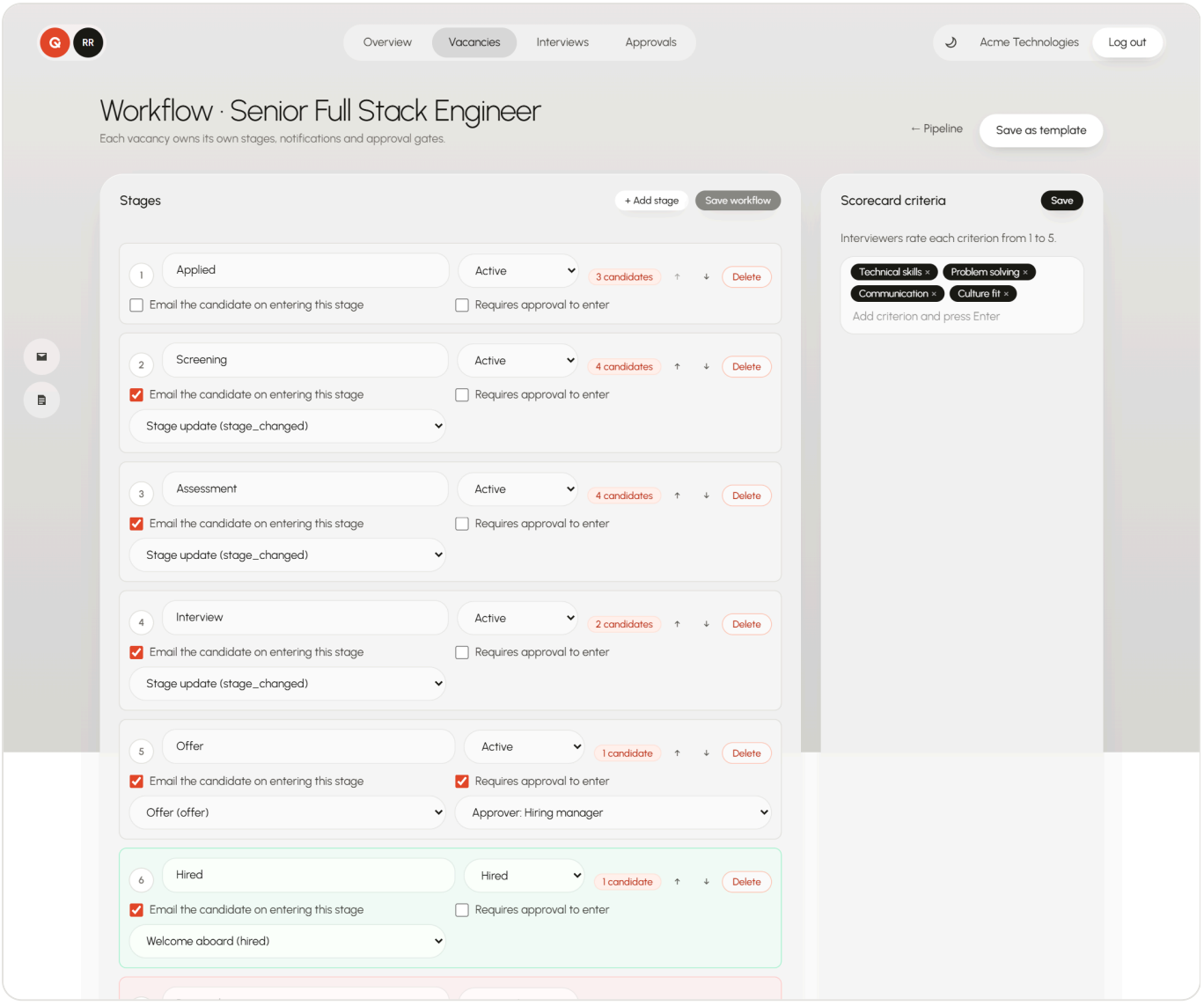
Title, department, location, employment type, hiring manager, salary range (min ≤ max is validated), description, requirements, skills, custom application questions, and a pipeline template. New jobs start as drafts.

Pipeline Kanban `/jobs-admin/:id/pipeline`



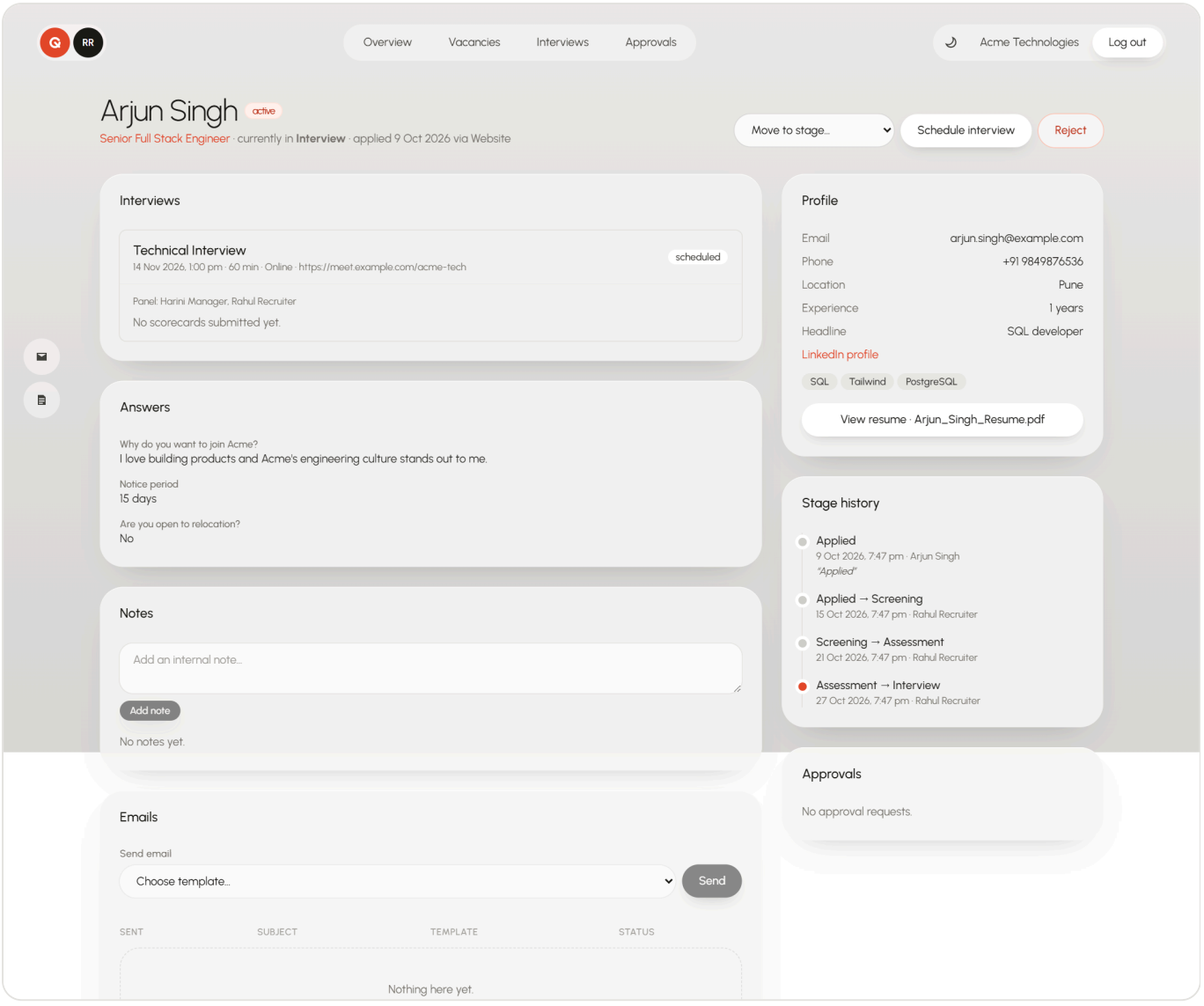
One column per stage of *this* vacancy. Cards show the candidate, their source, a skill-match ring and badges (interviews, approval pending), sorted best match first. Drag-and-drop or "Move to..." calls the pipeline engine; gated stages answer "Sent for approval".

Workflow editor ⭐ /jobs-admin/:id/workflow



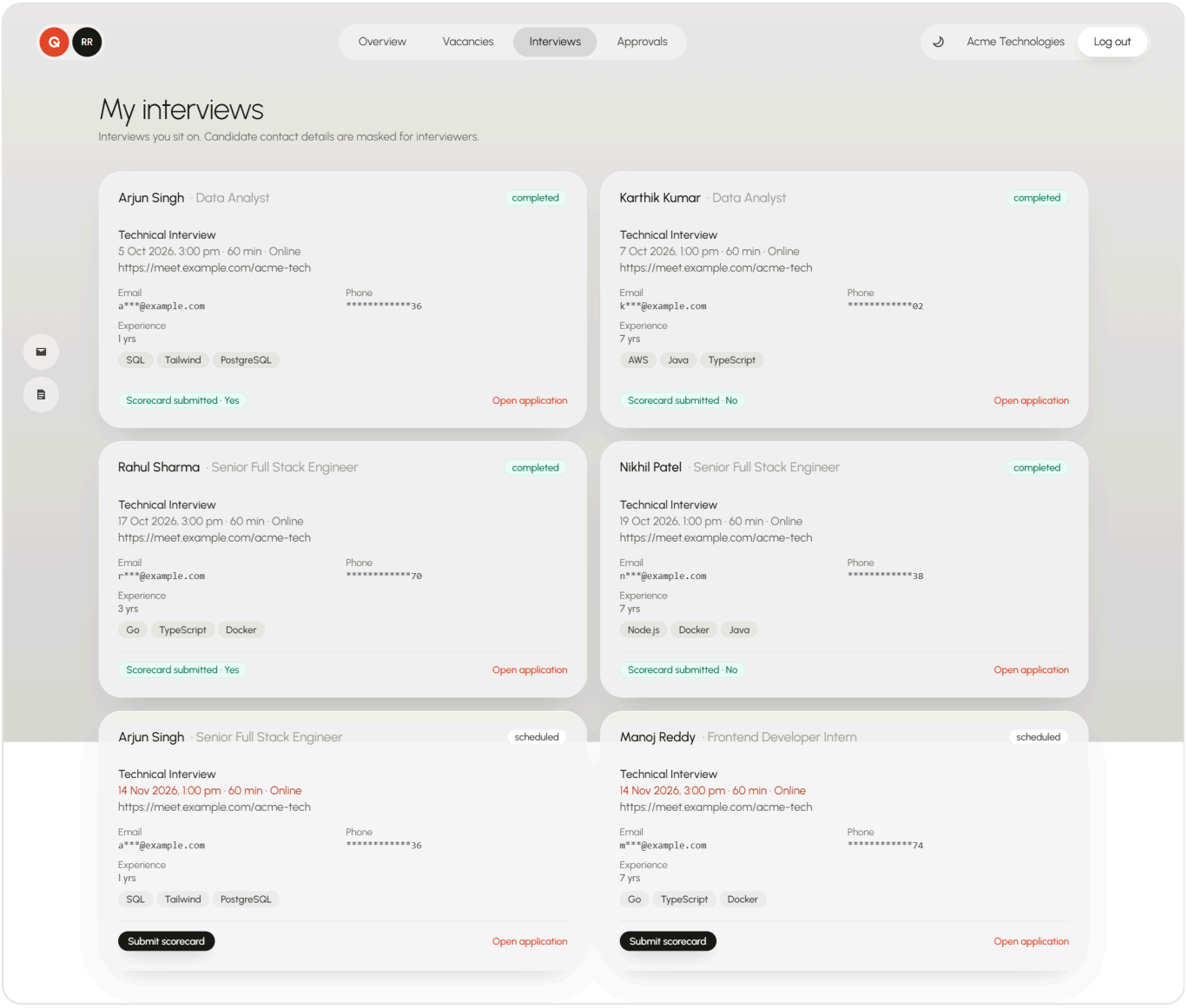
The first USP. Recruiters rename, reorder, add and remove stages; set each stage's category (active, hired or rejected); choose whether entering it emails the candidate and with which template; and set whether it needs approval and from which role. Scorecard criteria are edited here too, and any pipeline can be saved as a reusable template. Safety rules are enforced by the API: a rejected stage is required, names must be unique, and a stage that still holds candidates can't be deleted.

Candidate detail /applications/:id



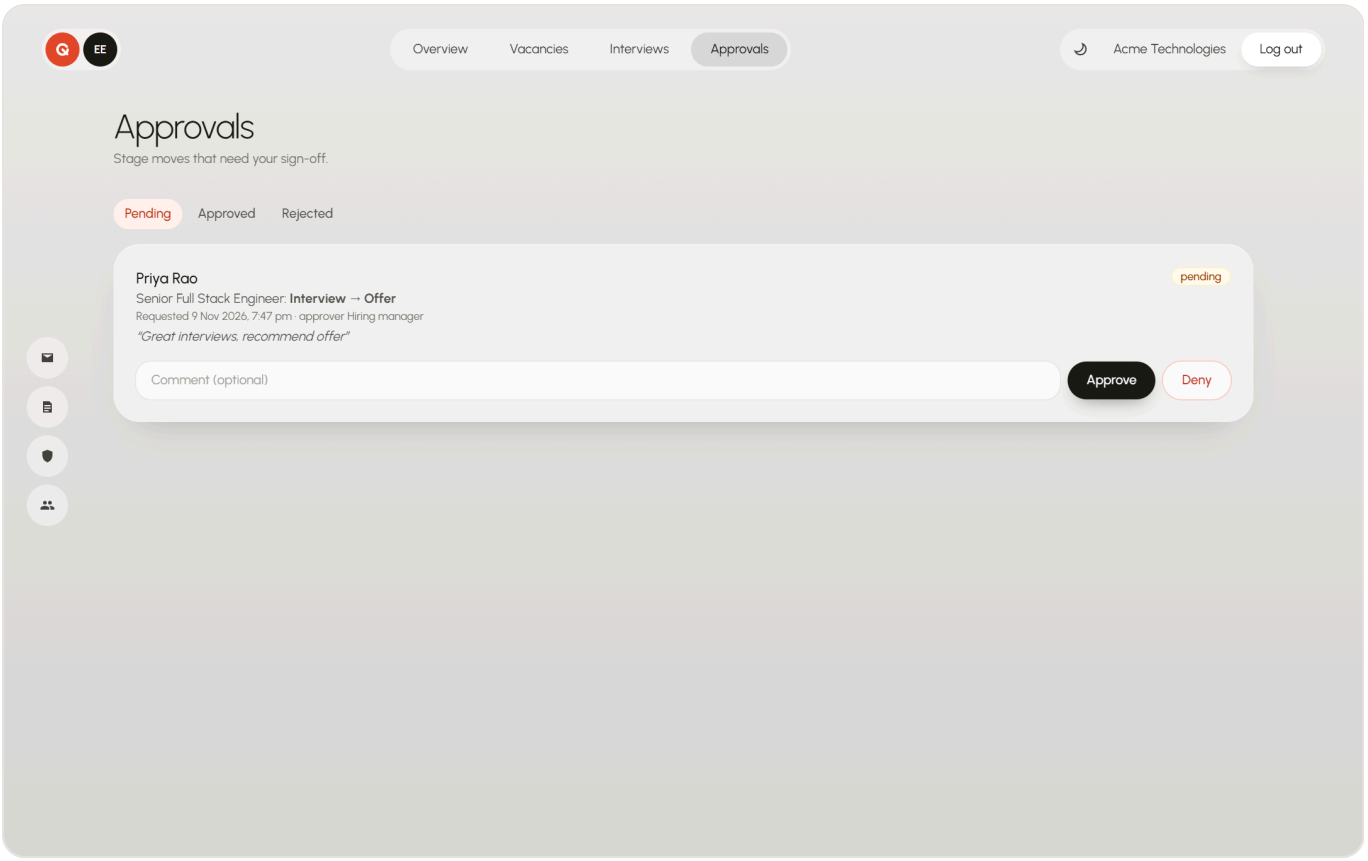
Everything about one application: profile and resume (each view is audited), answers, interviews with panels and scorecards (with an average per criterion), internal notes, approvals, the complete stage history with who moved the candidate and when, and the email log. From here recruiters can move or reject the candidate, schedule an interview or send a templated email.

My interviews & scorecards /interviews



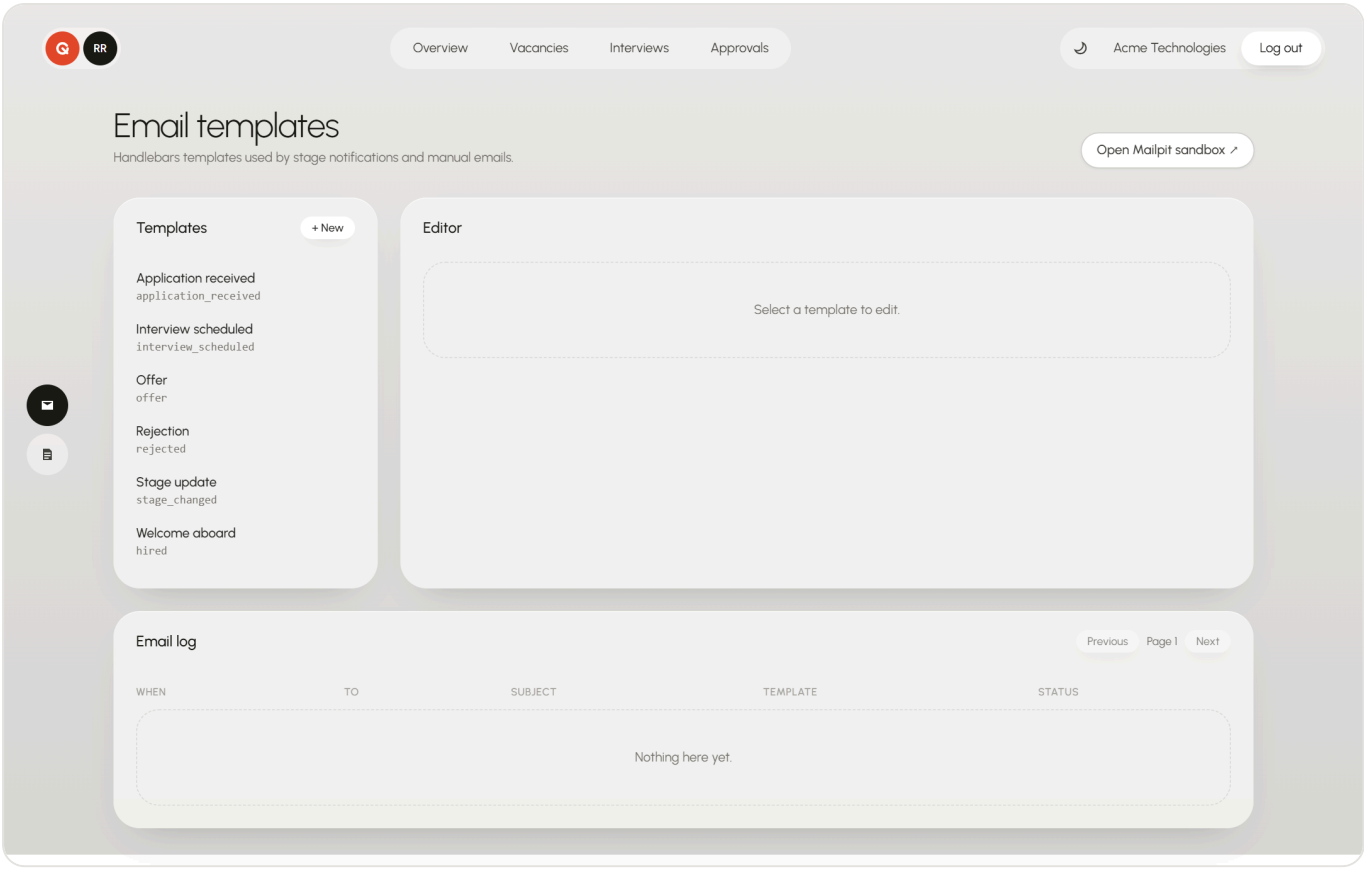
Each interviewer's queue. Candidate email and phone are masked (role-limited access). Scorecards require a 1–5 rating for every criterion of the job plus a recommendation, and only assigned panel members can submit them.

Approvals /approvals



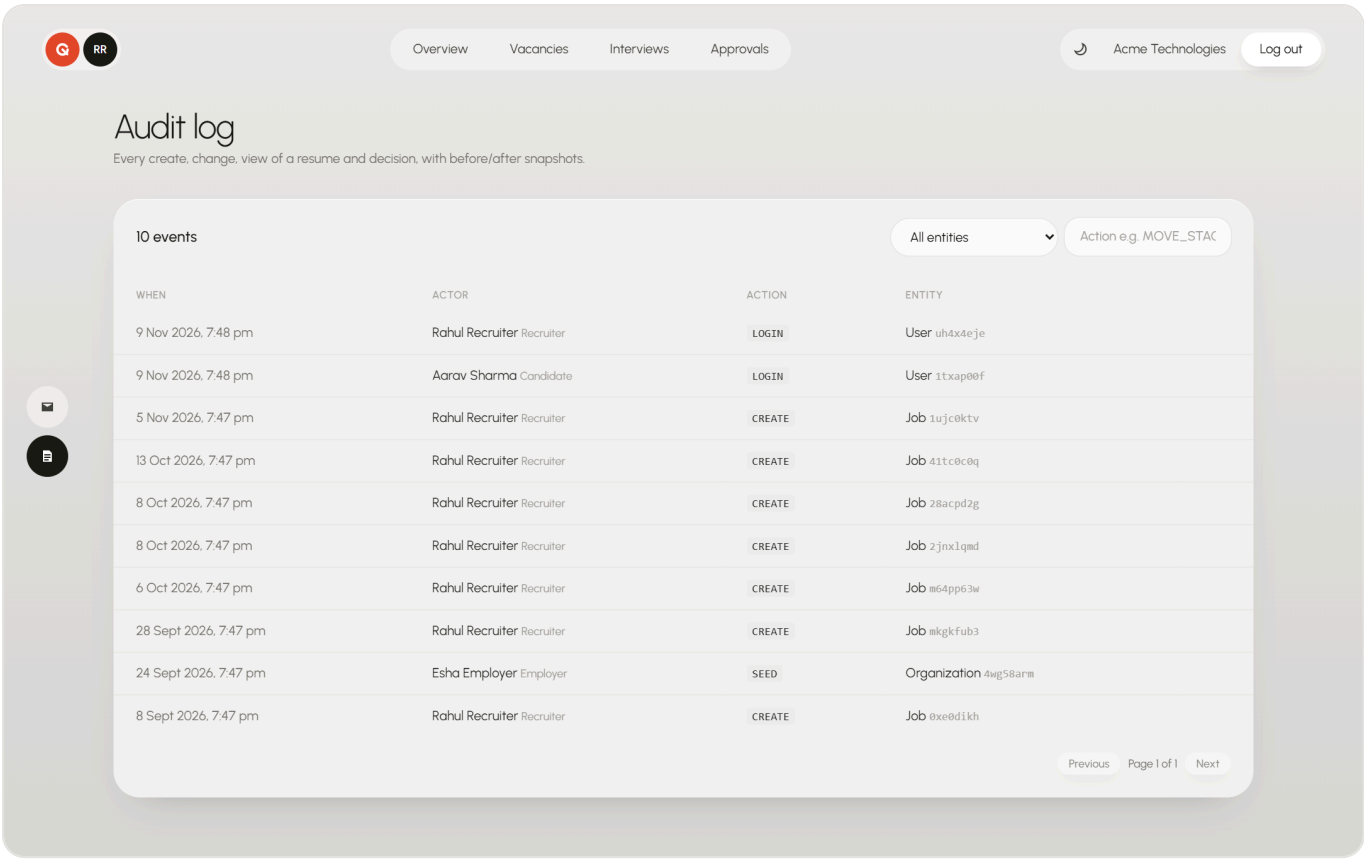
Pending stage moves that need a sign-off. The employer can approve anything; otherwise only the configured approver role can (and a hiring manager only on their own jobs). Approving completes the move and sends the stage's email; denying leaves the candidate where they are.

Email templates & log `/templates`



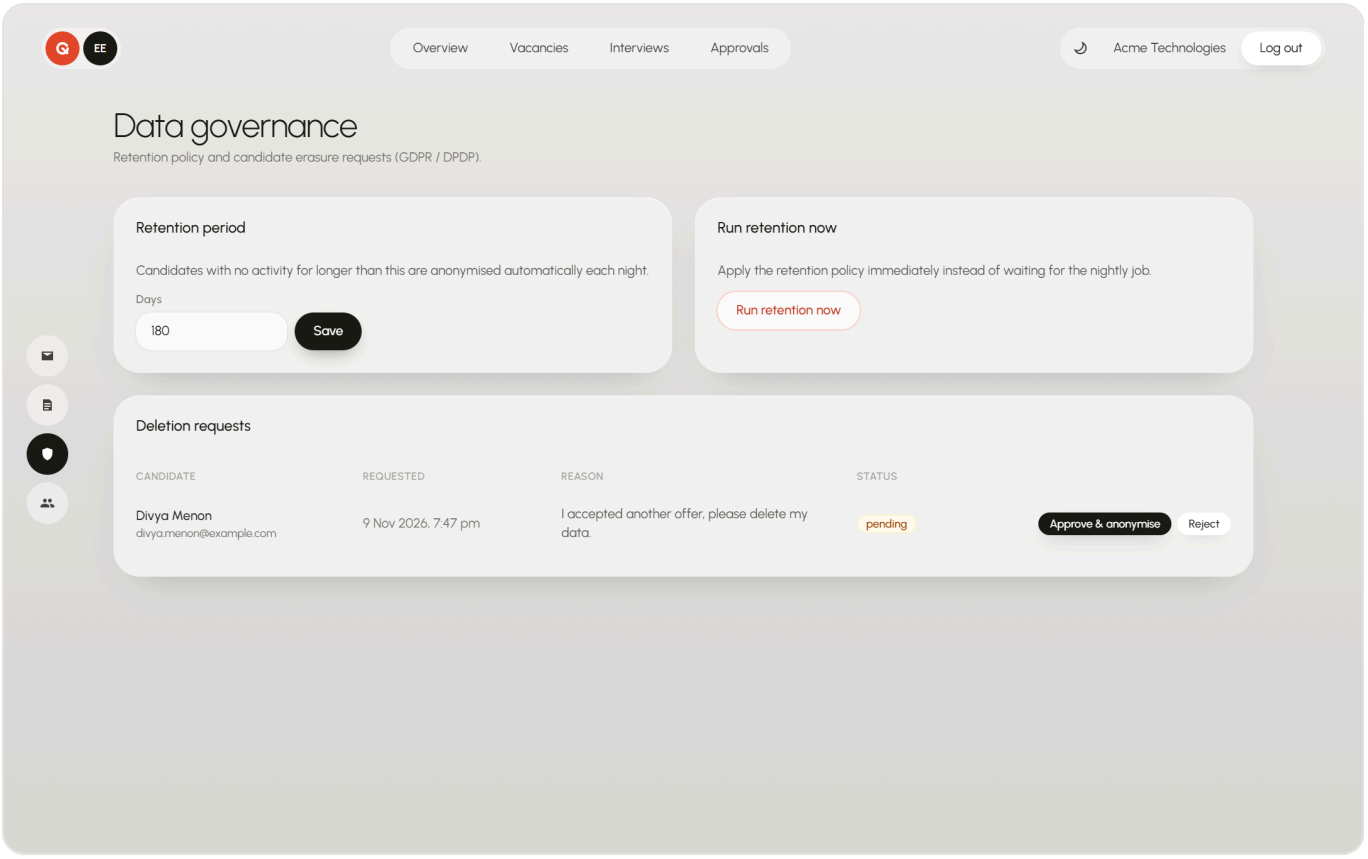
Editable templates with placeholders such as `{{candidateName}}`, `{{jobTitle}}` and `{{interviewDate}}`. Placeholder values are HTML-escaped. Every send is logged as queued, then sent or failed, and all mail lands in the Mailpit sandbox, never in a real inbox.

Audit log /audit



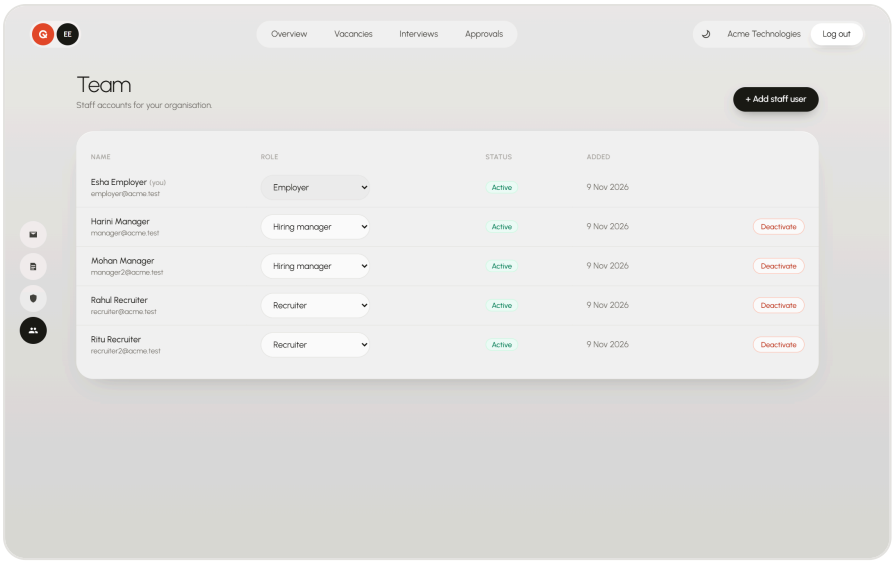
An append-only record of every write: who did it, in which role, what changed (before/after), on which record, from which IP and when. It's filterable by entity and action and paginated.

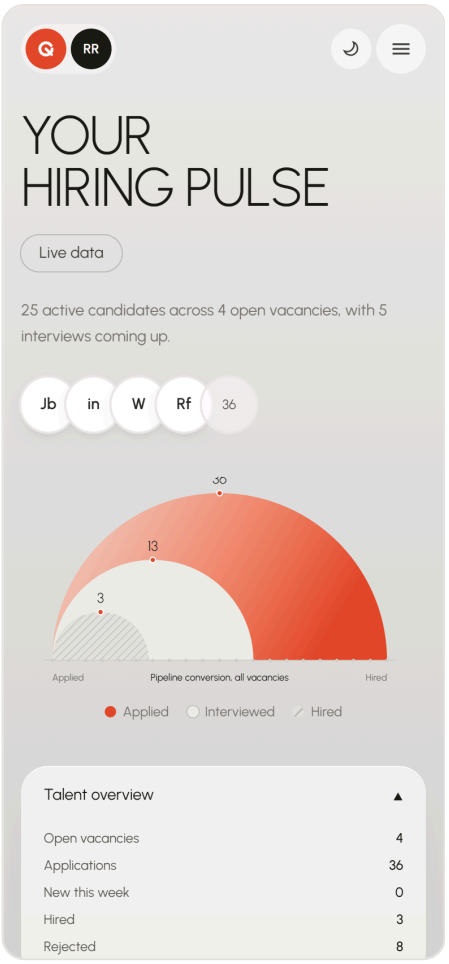
Data governance ⭐ /governance



The second USP. The employer sets the retention period, can run the retention sweep on demand (it also runs nightly), and reviews deletion requests. Approving a request anonymizes the candidate: name and email are replaced, contact fields cleared, answers and notes deleted, resume files removed from storage. Application rows and stage history are kept, so analytics remain correct.

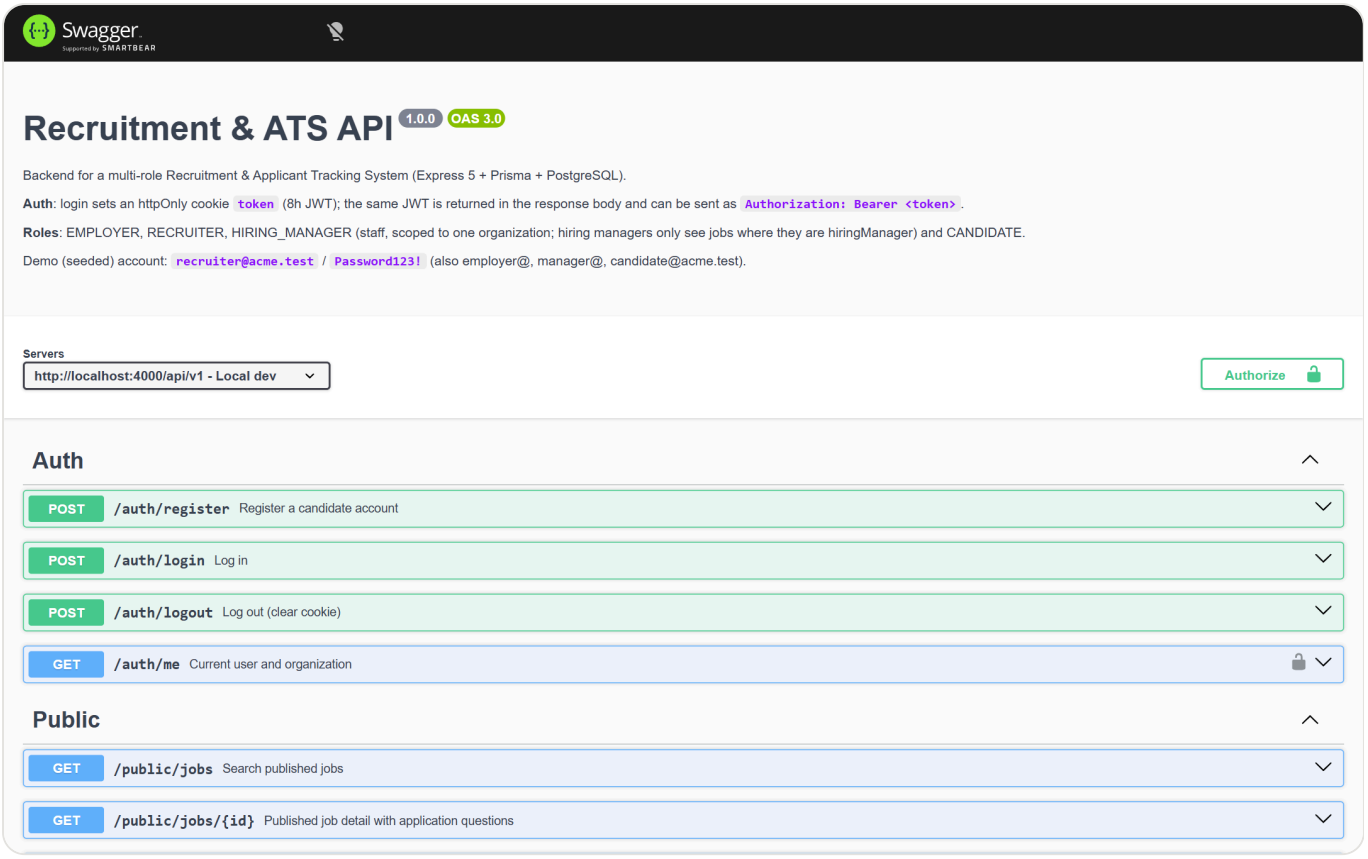
Team /users





The employer creates staff accounts, changes roles and deactivates people; deactivated users can no longer log in. Right: the dashboard on a phone.

API documentation :4000/api/docs



An OpenAPI 3 specification for all 62 operations, served by Swagger UI, with request schemas, the roles allowed per endpoint, error formats and examples (including the 202 approval-pending and 409 double-booking responses). A human-readable version with curl examples is in `docs/API.md`.

Configurable hiring workflow

- Every vacancy owns its ordered `JobStage` rows: name, order, category, notify flag, email template, requires-approval flag and approver role.
- Edits are diffed against the current stages inside a transaction. Stages still holding candidates can't be removed, and the rules (a rejected stage, unique names, an approver for every gate) are validated.
- The pipeline engine reads these rows at run time, so a new process (for example "Online Test → Tech Interview → Employer-approved Offer" for internships) needs no code.
- Pipelines can be saved as templates and chosen when creating a job.

Candidate data governance

- **Consent:** data-processing consent at sign-up, per-application consent, and optional talent-pool and marketing consent, each with policy version, IP and timestamp; withdrawals are kept in the history.
- **Role-limited access:** interviewers see masked contact details, hiring managers see only their jobs, and every staff resume view is audited.
- **Retention:** candidates whose every application in the organisation is rejected or withdrawn and older than the retention period are anonymized automatically.
- **Export & deletion:** self-service JSON export, and deletion requests with employer review, as described on the governance page.

8 · DATA MODEL

21 models, built for traceability

Group	Models	Key rules
Identity	Organization, User, CandidateProfile, Resume, Consent	Unique email; profile 1:1 with user; cascade on delete
Jobs	Job, JobQuestion, JobStage, ScorecardCriterion, PipelineTemplate	Index on (org, status) and (job, order)
Pipeline	Application, ApplicationAnswer, StageTransition, StageApproval, Note	Unique (job, candidate); index on (job, current stage); append-only transitions
Interviews	Interview, InterviewPanel, Scorecard	One scorecard per interviewer per interview
Comms & audit	EmailTemplate, EmailLog, Notification, AuditLog, DataRequest	Unique (org, template key); audit indexed by entity and time

Full ER diagram, constraints and indexes: docs/DATABASE.md.

9 · RESUME PARSING & REAL-RESUME DEMO DATA

Reading resumes on the server

PDF text is extracted with `unpdf`. Heuristics then find the email, phone, LinkedIn URL, city and headline, and match against a list of about 40 skills to produce canonical skill tags that line up with job requirements. Nothing is sent to an outside service.

A bulk importer and a resume-based seed mode turned a batch of 26 real student resumes into candidates: each was parsed, stored, applied to the best-fitting open role, and placed in that role's pipeline by skill match (hired, offer awaiting approval, interviewing with scorecards, assessment, rejected). 24 of 26 parsed fully; 2 scanned PDFs without a text layer were imported with name and file only. The real resumes are gitignored and never published; all screenshots in this report use generated demo data.

10 · SECURITY CONSIDERATIONS

Defence in depth

- **Authentication:** bcrypt hashes; JWT in an httpOnly, SameSite=Lax cookie with 8-hour expiry; login rate-limited; deactivated users rejected.
- **Authorization:** role middleware plus resource checks in one module; hiring-manager scope applied inside SQL; tests prove the 403s.
- **Input:** Zod on every body and query; strict candidate profile schema; validation errors returned as 400s with per-field details.
- **Files:** MIME allow-list and 5 MB limit; private bucket; 5-minute presigned reads; resume views audited.
- **Transport & headers:** helmet, CORS allow-list, 1 MB JSON limit.
- **Data:** HTML-escaped email templates, masked PII for interviewers, anonymization on deletion, no secrets in Git.

11 · TESTING & RESULTS

74 automated tests, all passing

Suite	Tests	Covers
Backend unit	25	Job status rules, access rules, PII masking, stage validation, approvals, interview overlap, template escaping
Auth & RBAC	11	Register / login / logout, 401s, 403s, IDOR, inactive users
Jobs & validation	8	Salary rules, required answers, strict profile, status machine, duplicate and paused applications
Pipeline & workflow	10	Moves, cross-job guard, approval gate (202), deny, reject, stage CRUD rules, audit row
Interviews	7	Scheduling, past date, double-booking 409, scorecard permissions and validation, PII masking
Governance & uploads	6	Export, deletion approve/reject, retention, audit access, PDF/TXT upload
Frontend	7	API client errors and status codes, role guards, status transitions

Integration tests run the real Express app against a separate PostgreSQL test database, which is wiped and re-seeded before every test, with real S3 storage and SMTP. Backend line coverage is 74%. GitHub Actions runs type checks and both suites on every push.

12 · LIMITATIONS & FUTURE ENHANCEMENTS

What's next

Limitation today	Planned improvement
Email is sent without a retry queue	Move to a job queue (e.g. BullMQ) with retries and delivery webhooks
8-hour session without refresh tokens	Rotating refresh tokens and session revocation
Skill extraction is keyword-based; scanned PDFs need OCR	OCR for image-only resumes; semantic matching with embeddings
No calendar integration	Google / Outlook calendar sync and candidate self-scheduling
Org-wide funnel groups stages by name	Normalised stage categories for cross-pipeline analytics
No browser end-to-end test suite	Playwright journeys for the main flows
Single organisation in practice	Full multi-tenant isolation, SSO, billing

13 · RUNNING THE PROJECT

Setup in three commands

```
docker compose up -d db storage mailpit
```

```
cd backend && cp .env.example .env && npm install && npx prisma migrate deploy && npm run db:seed && npm run dev
```

```
cd frontend && npm install && npm run dev
```

Address	What
http://localhost:5173	Web app
http://localhost:4000/api/docs	Swagger API docs
http://localhost:8025	Mailpit sandbox inbox

Demo logins (password `Password123!`): employer@acme.test · recruiter@acme.test · manager@acme.test · candidate@acme.test

Source, documentation and this report: github.com/christhecreator56/hireflow